

Прийоми ефективного програмування

Частина 3. Менеджери контексту

Менеджер контексту, який задається ключовим словом `with`, використовувався під час роботи з файлами. Фактично за інструкцією `with` захований функціонал блоку `try-finally`. Так, код

```
with open('data.txt') as file:
    data = file.read()
```

еквівалентний такому:

```
try:
    file = open('data.txt')
    data = file.read()
finally:
    file.close()
```

Користувач може писати власні менеджери контексту у вигляді класів, але у багатьох випадках значно простіше декорувати функцію декоратором `@contextmanager` із модуля `contextlib` стандартної бібліотеки Python. Інструкція `yield` у тілі функції розділяє код на дві частини: код до неї відповідає блоку `try`, а код після неї – блоку `finally`.

Припустимо, нам потрібно у багатьох місцях програми перемикатися на якусь директорію (папку), виконувати у ній певні дії, та виходити з неї (повертатися до робочої директорії). Тоді може стати у пригоді, наприклад, такий менеджер контексту:

```
import os
from contextlib import contextmanager

@contextmanager
def visit_directory(directory):
    old_directory = os.getcwd()
    os.chdir(directory)
    yield
    os.chdir(old_directory)
```

Використаємо цей менеджер контексту для додавання папки `NEW` у кожену папку, яка міститься у робочій директорії:

```
paths = [item for item in os.listdir()
         if os.path.isdir(item)
         and not item.startswith('.')]

for item in paths:
    with visit_directory(item):
        os.mkdir('NEW')
```

Наш менеджер контексту нічого не повертає, він просто передає керування – тому ми і не використовуємо `as` після `with`. Якщо нам потрібно мати доступ до якогось об'єкта у тілі інструкції `with`, то цей об'єкт потрібно повертати інструкцією `yield`.